Flutter Topics

Context

1. Flutter Introduction

2. Types Of Widgets (1st)

(a) Single Child Widget
(b) Multiple Child Widget
(c) Leaf Widget

3. Types Of Widgets (2nd)

(a) Stateless Widget
(b) Stateful Widget
(c) Inherited Widget

4. Types Of Widgets (3rd)

(a) High-Level Widgets
(b) Medium Level Widgets
(c) Base Level Widgets

5. Types Of Widgets (4th)

(a) Box Widgets
(b) Sliver Widgets

6. Basic Widgets

1. Text Widget
2. MaterialApp Widget
3. Cupertino Widget
4. Scaffold Widget
5. Container Widget

7. UI Alignment Widgets

1. Align Widget
2. Center Widget
3. SizedBox Widget
4. Padding Widget
5. Row Widget
6. Column Widget
7. Stack Widget
8. Positioned Widget
9. Opacity Widget

8. Constrained Widgets

1. ConstrainedBox Widget
2. UnconstrainedBox Widget
3. OverflowBox Widget
4. LimitedBox WIdget
5. FittedBox Widget

9. Responsive Widgets

1. Flexible Widget
2. Expanded Widget

10. Scaffold Widgets

1. FloatingActionButton Widget
2. AppBar Widget
3. NavigationDrawer Widget
4. ListTile Widget
5. ButtonBar Widget
6. BottomNavigationBar WIdget

11. Event Listener Widgets

1. InkWell Widget
2. GestureDetector Widget
3. Builder Widget
4. LayoutBuilder Widget

12. Input Widgets

1. TextField Widget
2. TextFormField Widget
3. Form Widget
4. CheckBox Widget
5. Radio Widget
6. AutoComplete Widget

13. Asset Widgets

1. CircleAvatar Widget
2. Image Widget

14. Decoration Widgets

1. BoxDecoration Widget
2. DecoratedBox Widget
3. ShapeDecoration Widget
4. InputDecoration Widget

15. TextEditingController Widgets

1. TextEditingController Widget
2. FocusNode Widget

16. Key Widgets

1. Object Key
2. Value Key
3. Unique Key
4. Global Key

17. Clipper Widgets

1. ClipOval Widget
2. ClipRect Widget
3. ClipRRect Widget
4. ClipPath Widget

18. Paint Widgets

1. CustomPaint Widget
2. TextPainter Widget
3. TextSpan Widget

19. Animations

Types Of Animation

(a) **Implicit Animation**

Implicit animation uses the built-in widgets provided by the flutter in which we are simply animating between two points. A few implicit animation widgets are:

(i) AnimatedOpacity Widget
(ii) AnimatedPositioned Widget
(iii) AnimatedContainer Widget
(iv) AnimatedSize Widget

(v) AnimatedScale Widget

In these implicit widgets, two parameters are generally required.
One - duration
Two - widget's property
So, the compiler animates that specific property from default to a given value in the given duration. That specific widget's property is generally applied to the child widget.

```
AnimatedScale(
  scale: _scale,
  duration: const Duration(seconds: 5),
  child: Container(
    width: 30,
    height: 30,
    color: Colors.green,
  ), // Container
), // AnimatedScale
```

For example in this Implicit Animated widget, we have a container as a child, as the value of scale will change to a new value the animated widget will animate the container from the previous value to a new value in 5 seconds as given.

Another implicit widget that is important to discuss separately is the

AnimatedCrossFade Widget :

This widget additionally requires the first child (widget) and second child (widget) to animate from one child to another in the given duration.

Another way to animate is a little bit more flexible is by using the **TweenAnimationBuilder Widget**. Firstly let's understand what Tween Animation is**,** animation of any property between two values (begin value, end value) is called Tween Animation.

```
TweenAnimationBuilder(
  tween: Tween<double>(begin: begin, end: end),
  duration: const Duration(seconds: 3),
  builder: (context, value, child) {
    return Transform.rotate(
      angle: value,
      child: child,
    ); // Transform.rotate
  },
  child: Container(
    width: 100,
    height: 100,
    color: Colors.green,
  ), // Container
) // TweenAnimationBuilder
```

In this widget, the required parameters are
(i) **Tween :** (Object contains two values (begin value, end value) of any data type)
(ii) **Duration :** (duration in which animation will complete between two values)
(iii) **Builder :** (of delegate type) contains a value at the instance, child of the widget, and context which every builder has. In this builder, we must return a widget with the child which will get painted in every frame with the changing value.

(b) **Explicit Animation**

Explicit Animation involves complex animation like drawing-based animations built by the user.

I.Built-in Explicit Animation :

In this animation we have control over the animation however we will still use a built-in widget for animation i.e. **ScaleTransition()**.

```
class _ImplementationOfTransitionWidgetState
    extends State<ImplementationOfTransitionWidget>
    with SingleTickerProviderStateMixin {
  late AnimationController scaleAnimationController;
  @override
  void initState() {
    super.initState();
    scaleAnimationController =
        AnimationController(vsync: this, duration: const Duration(seconds: 10));
    scaleAnimationController.forward();
  }

  @override
  Widget build(BuildContext context) {
    return Center(
      child: ScaleTransition(
        scale:
            Tween<double>(begin: 0, end: 1).animate(scaleAnimationController),
        child: Container(
          width: 100,
          height: 100,
          color: Colors.green,
        ), // Container
      ), // ScaleTransition
    ); // Center
  }
}
```

II.Fully Explicit Animation :

In this animation, we have complete manual control over the animation. We use addListener from the controller to manually setstate() at every frame. i.e.

```dart
class _ImplementationOfTransitionWidgetState
    extends State<ImplementationOfTransitionWidget>
    with SingleTickerProviderStateMixin {

  late AnimationController scaleAnimationController;
  late Animation<double> scaleAnimation;

  @override
  void initState() {
    super.initState();
    scaleAnimationController =
        AnimationController(vsync: this, duration: const Duration(seconds: 10))..addListener(() { setState(() {}); });
    scaleAnimation = Tween<double>(begin: 0, end: 1).animate(scaleAnimationController);
    scaleAnimationController.forward();
  }

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Transform.scale(
        scale: scaleAnimation.value,
        child: Container(
          width: 100,
          height: 100,
          color: Colors.green,
        ), // Container
      ), // Transform.scale
    ); // Center
  }
}
```

III. Curves in Animation :

In this animation we can create a curve over the timeline of animation i.e. animation could be slower at the start and gradually becomes faster. We can use this curved animation as a parameter in the animate() tween method.

In **CurvedAnimation** object, it requires two parameters,

(i) parent (which will be the controller)

(ii) curve (we use the constants in the curve class to generate curves in animation)

```dart
class _ImplementationOfTransitionWidgetState
  @override
  void initState() {
    super.initState();
    scaleAnimationController =
        AnimationController(vsync: this, duration: const Duration(seconds: 10))
          ..addListener(() {
            setState(() {});
          });
    scaleAnimation = Tween<double>(begin: 0, end: 1).animate(
      CurvedAnimation(
        parent: scaleAnimationController, curve: Curves.bounceInOut),
    );
    scaleAnimationController.forward();
  }

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Transform.scale(
        scale: scaleAnimation.value,
        child: Container(
          width: 100,
          height: 100,
          color: Colors.green,
        ), // Container
      ), // Transform.scale
    ); // Center
  }
}
```

IV. Staggered Explicit Animation :

In this animation we have created a scene using the controller, while giving the controller as a parameter to the animate method of a tween, we use the **Interval** object as a parameter in place of a curve.

In Interval, it requires two must parameters (begin: end:)

(it also has an optional curve parameter)

```dart
class _ImplementationOfTransitionWidgetState
@override
void initState() {
  super.initState();
  scaleAnimationController =
      AnimationController(vsync: this, duration: const Duration(seconds: 20))
        ..addListener(() {
          setState(() {});
        });
  scaleAnimation = Tween<double>(begin: 0, end: 1).animate(
    CurvedAnimation(
        parent: scaleAnimationController,
        curve: const Interval(0, 0.6, curve: Curves.bounceInOut)), // CurvedAnimation
  );
  colorAnimation = ColorTween(begin: Colors.green, end: Colors.blue).animate(
    CurvedAnimation(
        parent: scaleAnimationController,
        curve: const Interval(0.5, 1, curve: Curves.bounceInOut)), // CurvedAnimation
  );
  scaleAnimationController.forward();
}

@override
Widget build(BuildContext context) {
  return Center(
    child: Transform.scale(
      scale: scaleAnimation.value,
      child: Container(width: 100, height: 100, color: colorAnimation.value),
    ), // Transform.scale
  ); // Center
}
}
```

V.Tween Animation Builder :

Tween Animation Builder simply uses tween object, duration, and builder in builder it implicitly gives the value that we use in the desired animation property, the value keeps changing according to the tween and duration.

```dart
@override
Widget build(BuildContext context) {
  return TweenAnimationBuilder(
    tween: ColorTween(begin: Colors.green, end: Colors.blue),
    duration: const Duration(seconds: 3),
    builder: (context, value, child) {
      return Container(
        width: 100,
        height: 100,
        color: value,
      ); // Container
    },
  ); // TweenAnimationBuilder
}
}
```

VI.Animation Builder :

We use this animation to create minimal animations like changing borders, changing colors, etc. It requires two parameters

(a) Animation<object>

(b) Builder (context, child){return widget}

```dart
class _ImplementationOfTransitionWidgetState
  late Animation<double> scaleAnimation;

  @override
  void initState() {
    super.initState();
    scaleAnimationController =
        AnimationController(vsync: this, duration: const Duration(seconds: 5))
          ..addListener(() {
            setState(() {});
          });
    scaleAnimation =
        Tween(begin: 1.0, end: 2.0).animate(scaleAnimationController);
    scaleAnimationController.forward();
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
    | animation: scaleAnimationController,
    | builder: (context, child) =>
    ├────Transform.scale(scale: scaleAnimationController.value, child: child),
    └child: Container(width: 50, height: 50, color: Colors.purple),
    ); // AnimatedBuilder
  }
}
```

VII. Custom Explicit Animated Widget :

In this animated widget. We use **AnimatedWidget** as a parent and override its build() method as well as constructor which requires a listenable object as a parameter.

```dart
class TextScaleWidget extends AnimatedWidget {
  final String text;
  const TextScaleWidget(
      {required super.listenable, super.key, required this.text});
  Animation<double> get scaleAnimation => listenable as Animation<double>;
  @override
  Widget build(BuildContext context) {
    return Transform.scale(scale: scaleAnimation.value, child: Text(text));
  }
}
```

We will use this widget in a stateful widget to animate the child using AnimationController. So the widget user wouldn't have to worry about animation, we can simply send the child widget.

```dart
class _TextScaleTransitionState extends State<TextScaleTransition>
    with SingleTickerProviderStateMixin {
  late AnimationController scaleController;
  @override
  void initState() {
    super.initState();
    scaleController =
        AnimationController(vsync: this, duration: const Duration(seconds: 5))
          ..repeat();
  }
  @override
  Widget build(BuildContext context) {
    return TextScaleWidget(
      listenable: Tween(begin: widget.scaleStart, end: widget.scaleEnds)
          .animate(scaleController),
      text: widget.text,
    ); // TextScaleWidget
  }
   @override
  void dispose() {
    scaleController.dispose();
    super.dispose();
  }

}
```

20. ListView

To display a list of items vertically or horizontally we use ListView Widget. ListView has differently named constructors.

I. ListView (Default)
It contains a property of children, which is simply scrollable in the UI.

II. ListView.builder
In this Constructor, the user gives the item count of how many times the builder is gonna generate and the second property is the item builder in which he receives the index number and context.

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      title: Text(widget.title),

    ), // AppBar
    body: ListView.builder(
      itemCount: 100,
      itemBuilder: (context, index) {
        return ListTile(
          leading: CircleAvatar(
            child: Text('$index'),
          ), // CircleAvatar
          trailing: const Icon(Icons.alarm),
        ); // ListTile
      },
    ), // ListView.builder
```

III. ListView.separated

It is similar to the builder but it also has a separator builder. The item builder works the same as in the builder but the separator builder re-runs after every execution of the item and builder and creates a separation.

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      title: Text(widget.title),

    ), // AppBar
    body: ListView.separated(
      itemBuilder: (context, index) {
        return ListTile(
          leading: CircleAvatar(
            child: Text('$index'),
          ), // CircleAvatar
          trailing: const Icon(Icons.alarm),
        ); // ListTile
      },
      separatorBuilder: (context, index) {
        return Container(
          width: 50,
          height: 50,
          color: Colors.blue,
        ); // Container
      },
      itemCount: 100), // ListView.separated
```

IV. ListView.custom

In this Constructor, the user has control over the list i.e. The user can make the list of his own choice based on his requirements. The user can achieve this by passing the relevant delegate in the **children delegate** attribute. To make the List behave like a simple List View pass the **SliverChildListDelegate** and the List behave like ListView.builder pass the **SliverChildBuilderDelegate**.

```
    body: Center(
      child: Column(
        children: [
          //Simple List View
          ListView.custom(
            childrenDelegate: SliverChildListDelegate(
              postsList,
            ), // SliverChildListDelegate
          ), // ListView.custom
          // Builder List View
          ListView.custom(
            childrenDelegate:
                SliverChildBuilderDelegate((context, index) => ListTile(
                      leading: CircleAvatar(
                        child: Text(
                          '$index',
                        ), // Text
                      ), // CircleAvatar
                      trailing: const Icon(
                        Icons.alarm,
                      ), // Icon
                    ))) // ListTile // SliverChildBuilderDelegate // ListView.custom
        ],
      )), // Column // Center
    ); // Scaffold
  }
}
```

21.GridView

Just like ListView, GridView is also used to make the items scrollable vertically or horizontally. The only difference is in GridView multiple items can be placed in the cross-axis. GridView also has named constructors.

I.GridView (Default)

It contains a property of **gridDelegate** in which the user passes the relevant delegate based on his requirements and children property in which all the children are simply scrollable across the relevant axis. Users can pass the following delegates :

(a) **SliverGridDelegateWithFixedCrossAxisCount** => Creates a delegate that makes grid layouts with a fixed number of tiles in the cross-axis. It ignores the size of the child to fulfill the number of items given in the cross-axis count.

```
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade300,
    body: Center(
      child: Column(
        children: [

          GridView(
            gridDelegate:
                const SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 4),
            children: const [],
          ), // GridView
        ],
      )), // Column // Center
    ); // Scaffold
  }
}
```

(b) **SliverGridDelegateWithMaxCrossAxisExtent** => Creates a delegate that makes grid layouts with tiles with a maximum cross-axis extent. It adjusts the number of children in the cross-axis according to the extent given.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade300,
    body: Center(
      child: Column(
        children: [
          GridView(
            gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent
            (maxCrossAxisExtent: 100),
            children: const [],
          ), // GridView
        ],
      )), // Column // Center
    ); // Scaffold
  }
}
```

II.GridView.extent

It is similar to GridView in which we pass SliverGridDelegateWithMaxCrossAxisExtent as the delegate type. In this constructor, we just have to pass the max cross-axis extent and the list of children to be displayed. If we want to add some changes like spacing between children in the cross axis and main axis we will use the Simple Grid View otherwise for simple arrangement we will use GridView.count.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade300,
    body: Center(
      child: Column(
        children: [
          GridView.extent(
            maxCrossAxisExtent: 100,
            children: const [],
          ) // GridView.extent
        ],
      )), // Column // Center
  ); // Scaffold
}
}
```

III.GridView.count

It is similar to GridView in which we pass SliverGridDelegateWithFixedCrossAxisCount as the delegate type. In this constructor, we just have to pass the number of cross-axis items and the list of children to be displayed. If we want to add some changes like spacing between children in the cross axis and main axis we will use the Simple Grid View otherwise for simple arrangement we will use GridView.count.

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade300,
    body: Center(
      child: Column(
        children: [
          GridView.count(
            crossAxisCount: 4,
            children: const [],
          ), // GridView.count
        ],
      )), // Column // Center
    ); // Scaffold
  }
}
```

IV.GridView.builder

In this constructor, we have to pass the gridDelegate type and we receive an item builder function with the parameters of context and the index, which gets called when a new item is generated. The user also gives the item count to generate the number of items through the builder.

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade300,
    body: Center(
      child: Column(
        children: [
          GridView.builder(gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent
          (maxCrossAxisExtent: 300,), itemBuilder: (context, index) {
            return Container(
              width: 100,
              height: 100,
              color: Colors.amber,
            ); // Container
          },) // GridView.builder
        ],
      )), // Column // Center
    ); // Scaffold
  }
}
```

V.GridView.custom

In this Constructor the user has full control over the GridView i.e. User can make the GridView based on his requirements. The User has to give the **childrenDelegate**(Whether the Grid View is simple or of builder type) and the **gridDelegate.**

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade300,
    body: Center(
      child: Column(
        children: [
          GridView.custom(
            gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(
              maxCrossAxisExtent: 300,
            ), // SliverGridDelegateWithMaxCrossAxisExtent
            childrenDelegate: SliverChildListDelegate(
              [],
            ), // SliverChildListDelegate
          ) // GridView.custom
        ],
      )), // Column // Center
  ); // Scaffold
}
}
```

22. Single Child ScrollView

Using this widget we can simply make a constant list of widgets scrollable in horizontal or vertical direction using row or column respectively. It becomes scrollable when the widgets exceed the screen limit.

```dart
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      title: Text(widget.title),
    ), // AppBar
    body: SingleChildScrollView(
      child: Column(
        children: [
          Container(
            height: 100,
            color: Colors.blueGrey,
          ), // Container
          Container(
            height: 100,
            color: Colors.green,
          ), // Container
          Container(
            height: 100,
            color: Colors.pink,
          ), // Container
          Container(
```

23. Sliver Protocol

The scrollable widgets are known as Sliver widgets and hence they use the Sliver protocol. The scrollable part of the UI is known as Sliver. Some of the Sliver Widgets similar to the Box Widgets are as follows :

Sliver Protocol Widgets

(i) Sliver App Bar
(ii) Sliver List
(iii) Sliver Grid
(iv) Sliver AnimatedGrid
(v) Sliver AnimatedList
(vi) Sliver FadeTransition

24. SliverToBoxAdapter

Some common box protocol widgets are not available as Sliver widgets i.e. if we want to add an image there is no Sliver widget available for that. To solve this problem we use the **SliverToBoxAdapter** Widget which converts the box Widget to SliverWidget.

```
Widget build(BuildContext context) {
  return CustomScrollView(
    slivers: [
    SliverToBoxAdapter(
      child: Image.asset(image),
    ) // SliverToBoxAdapter
```

25.SliverFillViewPort Widget

If we want our scrollable list to fill the whole screen we can use the SliverFillViewPort Widget. Each child on the list will fill the screen.

```
@override
Widget build(BuildContext context) {
  return CustomScrollView(
    slivers: [
    SliverFillViewport(delegate: SliverChildListDelegate([]),
      viewportFraction: 1.0,
      ), // SliverFillViewport
```

26.SliverFillRemaining Widget

We will use this Widget when we want to fill the remaining space on the viewport with a specific widget after using all the other slivers. This Widget will cover the remaining space and will let the child fill that space.

```
@override
Widget build(BuildContext context) {
  return CustomScrollView(
    slivers: [

    SliverFillRemaining(
      child: ListView(
        children: const [],
      ), // ListView
    ), // SliverFillRemaining
```

27. Navigation

One of the most common things in the app is the navigation from one page to another which Flutter achieves through the **Navigator** class. There are two types of navigation in Flutter:

(i) Navigator 1.0 (To navigate from one page to another)

(ii) Navigator 2.0 (Mostly used for Deep Linking)

(a) Navigator 1.0

There are several ways to navigate from one page to another:

1. UsingMaterialPageRoute

To Simply navigate from one page to another we can use the **Navigator. of(context).push()** method in which we pass the Material Page Route. This Widget has a required parameter which is the builder in which we have to pass the page to which we want to navigate to. For IOS use the CupertinoPageRoute.

```
ElevatedButton(
    onPressed: () {
      // Navigator.of(context).push(MaterialPageRoute(
      //   builder: (context) => const SecondPage(),
      // ));
```

Sending Arguments :

If we want to send Arguments we can do this by using the optional parameter **settings** which accepts the RouteSettings widget. RouteSettings Widgets has parameter **arguments** in which we pass the value that we want to send to the desired page.

```
ElevatedButton(
    onPressed: () {
      Navigator.of(context).push(MaterialPageRoute(
          builder: (context) => const SecondPage(),
          settings: const RouteSettings(arguments: 'Pakistan')));
```

Receiving arguments :

To receive the arguments we can use the **ModalRoute** which is an inherited widget. We cast the arguments into our required type and use it.

```dart
class SecondPage extends StatelessWidget {
  const SecondPage({super.key});
  static const pageName= '/second_page';

  @override
  Widget build(BuildContext context) {
    var args = ModalRoute.of(context)!.settings.arguments as String;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Page'),
      ),
      backgroundColor: Colors.blue,
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
             Text(args),
            ElevatedButton(onPressed: () {
              Navigator.of(context).pop();

            }, child: const Text('Finish'))
          ],
        ),
      ),
    );
  }
}
```

2. UsingPageRouteBuilder

If we want to add animation while navigating from one page to another we use the PageRouteBuilder widget. This Widget has a required parameter **pageBuilder** . This parameter accepts a function that has a context, animation of double, and secondary animation and returns a widget. The method to pass the arguments is the same as described earlier.

```
Navigator.of(context).push(PageRouteBuilder(
  pageBuilder: (context, animation, secondaryAnimation) =>
      const SecondPage(),
  transitionDuration: const Duration(seconds: 50),
  reverseTransitionDuration: const Duration(seconds: 50),
  transitionsBuilder:
      (context, animation, secondaryAnimation, child) {
    return SlideTransition(
      position: animation.drive(Tween<Offset>(
          begin: const Offset(-1, -1), end: Offset.zero)),
      child: RotationTransition(turns: animation,child: child,),
    );
  },
));
```

3. UsingRoutes

A better way to navigate is to use the routes in which we use the **Navigator.of(context).pushNamed()** method. This method requires the page name to which we want to navigate and also has an optional parameter for arguments.

```
// Navigator.of(context).pushNamed(SecondPage.pageName);
```

Material App has a parameter as **routes** in which we define the routes. This parameter accepts a Map<String, Widget Function(context)>. We define the pageName as String and the function is the same as the builder or pageBuilder methods in which we have to pass the page to which we want to navigate. If we use the routes method then we don't have to use the home parameter instead we have to define the initial route parameter in which we have to give the page we want to show first.

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    | routes: {
    ├──MyHomePage.pageName: (context) => const MyHomePage(title: 'hello'),
    └──SecondPage.pageName: (context) => const SecondPage(),
      },
      initialRoute: MyHomePage.pageName,
    ); // MaterialApp
  }
}
```

```
class SecondPage extends StatelessWidget {
  const SecondPage({super.key});
  static const pageName= '/second_page';
```

4. UsingOnGenerateRoute Method

The best way to perform navigation is to use the onGeneateRoute parameter in the Material App Widget. This parameter accepts a function that has a parameter of **RouteSettings** for the arguments and a return type of **Route<dynamic>.** There is also no need to define the home parameter instead we need to define the initial route parameter as described in the previous method.

```
Route<dynamic> onGenerateRoute(RouteSettings settings) {
  return switch (settings.name) {
    // MyHomePage.pageName => PageRouteBuilder(
    //     pageBuilder: (context, animation, secondaryAnimation) =>
    //         const MyHomePage(title: 'Home Page'),
    //   ),
    MyHomePage.pageName=> SlideLeftPageRoute(page:const MyHomePage(title: 'Home
Page')),
    // SecondPage.pageName => PageRouteBuilder(
    //     pageBuilder: (context, animation, secondaryAnimation) =>
    //         const SecondPage(),
    //     settings: settings),
    SecondPage.pageName=> SlideLeftPageRoute(page: const SecondPage(),settings:
settings),
    ThirdPage.pageName => PageRouteBuilder(
        pageBuilder: (context, animation, secondaryAnimation) =>
            ThirdPage(data: settings.arguments as String),
      ),
    _ => PageRouteBuilder(
        pageBuilder: (context, animation, secondaryAnimation) =>
            const ErrorPage(),
      )
  };
}
```

```dart
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      initialRoute: MyHomePage.pageName,
      onGenerateRoute: onGenerateRoute,
    );
  }
}
```

28. Navigation 2.0 (Pending)
29. Go Router (Pending)
30. APIs

API (Application Programming Interface) is an interface that is used to interact with the data present in the server with security. APIs make sure that developers or anyone who is working at the front end are not directly interacting with the databases or server.

There are different methodologies by which different companies use APIs but the latest and most used technique is using RESTful APIs. As a prerequisite let's understand a few terms :

URL (URL is nothing but a URI)
URI (URI is a syntactical string that refers to a particular location, in URL that location is on the network)

Now let's understand how a RESTful API works,

RESTful APIs work on request-response. The developer generates a request for a particular event. This request includes the URL where the data exists. Now let's break down how an API created.

https://jsonplaceholder.typicode.com/posts/1

Base address: https://jsonplaceholder.typicode.com/
This is the base address in the given API which means the data belongs to this website.
API address: posts/
This is the API address where the data is present as we know a website contains a lot of pages so such a page that is dedicated to the JSON file (API) we considered it API address.

Endpoints: 1
Most of the APIs are lists so sometimes we don't want the entire list but a specific item from the list for that purpose we use endpoints to extract the exact item rather than fetching the entire list. But we can't use the endpoints on our own. If the API supports the endpoints then we can use or we can request the API developer to include endpoints.

Now we know how an API request works. Now we'll understand how to implement an API in flutter.

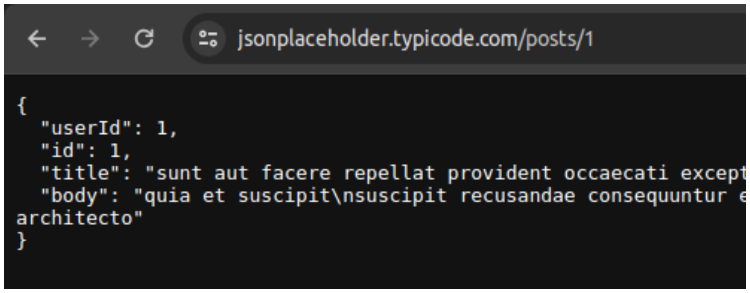
First and the most important thing is to understand the API structure, let's do that!

API structure :

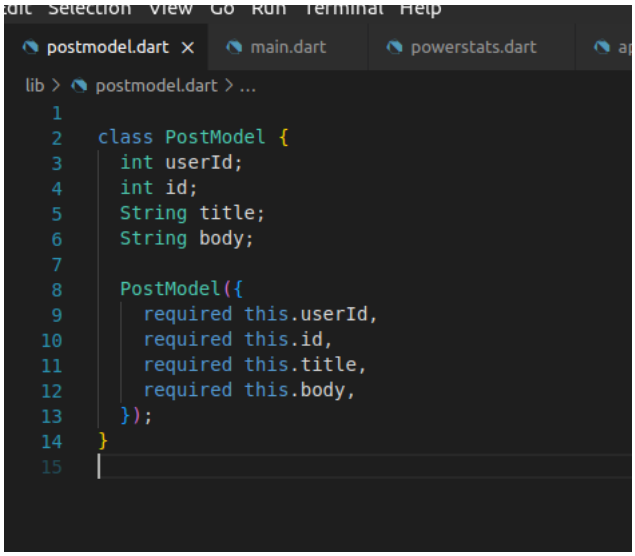
API is basically a json file. In flutter we can understand it as a List of Map<String,dynamic>. As we discussed API is generally a list made up of items. An item is a Map made up of **String** keys and **dynamic** values.
Here Dynamic values means values with different data types so we can't generalize it. It is also possible that in an item a value consists of a further list so we have to keep that in mind too.

Now, the first thing we'll do is to create a model class against the item.

Let say the item looks like this:



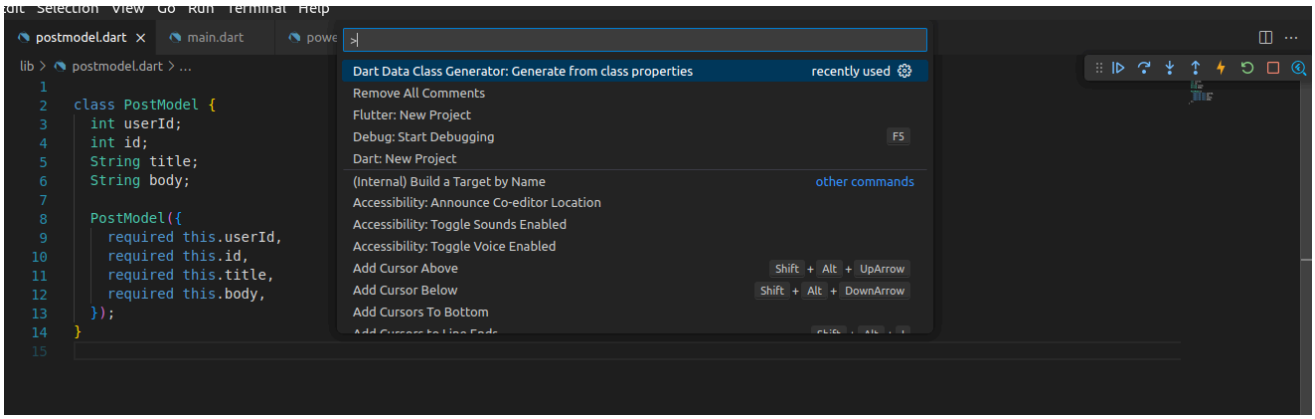
So the model class will be more or less like this:



Now, here's a catch we need a few methods that will be used frequently, like,

Json to Map	(fetching item from the API as JSON and converting to map)
Map to ModelClass	(converting the map into the model class's object)
ModelClass to Map	(converting the model class's object to map)
Map to JSON	(converting the map to JSON and pushing to API)

We can create these methods on our own, for json we can use json.decode() and json.encode(), for the map, we can manually create an object and put the values in every attribute and vice versa. but we can also use the extension Data Class Generator which will do all of this.



This will do all the work on our behalf.

Now we are ready to fetch data from the API into our model object and consume it.

By importing the "**http**" package, we'll receive the method named get().
By simply passing the complete URL we'll get the list in the Future. If we include the endpoint then we'll receive a single item but we have to make sure that the API developer or backend developer allows us to do so.

but we will do it with a proper structure so that in the future if the base address or API address gets changed we don't have to change it in a dozen places.

main.dart | fake_api.dart | postmodel.dart

lib > fake_api.dart > ...

```dart
import 'package:flutter_api_demo_implementation/postmodel.dart';
import 'package:http/http.dart';

extension APIextension on Response {
  bool get isSuccessful => statusCode == 200;
}

class FakeAPI {
  static const String baseUrl = 'https://jsonplaceholder.typicode.com';
  static const String postApiUrl = '/posts/';
  int endPoint;

  FakeAPI({required this.endPoint});

  Future<PostModel?> fetchPost() async {
    var response =
        await get(Uri.parse(baseUrl + postApiUrl + endPoint.toString()));
    if (response.isSuccessful) {
      return PostModel.fromJson(response.body);
    }

    return null;
  }
}

```

Now we must give it an endpoint while creating its object, so it will fetch a single post at a time. That's how we fetch the data now the data will come in the future so we'll handle this using FutureBuilder().

```dart
class _MyHomePageState extends State<MyHomePage> {
  late TextEditingController idController;
  Future<PostModel?>? postModel;
  void _incrementCounter() {
    setState(() {
      postModel = FakeAPI(endPoint: 1).fetchPost();
    });
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
    ├─appBar: AppBar(
    │ │ backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    │ └─title: Text(widget.title),
    │ ), // AppBar
    ├─body: Center(
    │ └─child: FutureBuilder(
    │   │ future: postModel,
    │   │ builder: (context, snapshot) {
    │   │   if (snapshot.connectionState == ConnectionState.none) {
    │   ├────return const Text('Click to load Data');
    │   │   } else if (snapshot.connectionState == ConnectionState.active ||snapshot.connectionState == ConnectionState.waiting) {
    │   ├────return const CircularProgressIndicator(color: Colors.green);
    │   │   } else if (snapshot.connectionState == ConnectionState.done) {
    │   ├────return Text(snapshot.data!.title);
    │   │   } else {
    │   └────return const Text('data is not fetched properly');
    │       }
    │     },
    │   ), // FutureBuilder
```

In the future builder the snapshot contains the object as well as ConnectionState so we can shift widgets according to the ConnectionState of future.
In this example, we've only fetched the title from the model object as we can also use other information too. This is how simply we can fetch data from API.

31. Local Database (SQLite)

A Local Database is used to store the data locally in tabular form mostly. SQLite is a non-procedural language that is used to create databases locally as well as to create tables and perform CRUD operations. (Create, Read, Update, Delete).

One of the most popular packages for local databases is **sqflite**. To implement a local database we have to import the following packages into our project.

```
sqflite: ^2.3.3+1
path: ^1.9.0
```

The **path** package provides functions and classes for storing the database on the disk.
The **sqflite** package provides functions and classes to interact with the database.

First, we will define the model classes which we will consume in the database. Make sure to name all the data members as per the attributes of the database table. As the sqflite insert method and many more methods request map<attribute, value> to execute it will be easy to convert an object into a map.

Further, by using a data class generator extension, we will create all the model class required methods.

Now we will create the database in the local storage using the openDatabase() function available in the sqflite package.

As the only required parameter we will give it the path using the getDatabasesPaths() method concatenated will our database name. This will return us Database object as Future<Database>.

Now we will add the optional parameter onCreate and give it the closure. That closure provides us with the database version and parameters. Now we will create the tables as per our schema in that onCreate function. database.execute() allows us to execute any SQL command on the database at any time.

This is how it will work,

```
late Database database;

static const dogsTableName = 'dogs';
static const catsTableName = 'cats';

final String dogsTableCreateCommand = 'CREATE TABLE $dogsTableName(id INTEGER PRIMARY KEY,name TEXT,weight REAL,price REAL)';
final String catsTableCreateCommand = 'CREATE TABLE $catsTableName(id INTEGER PRIMARY KEY,name TEXT,weight REAL,price REAL)';

void createDatabase() async {
  database = await openDatabase(
    join(await getDatabasesPath(), 'pets.db'),
    onCreate: (db, version) {
      db.execute(catsTableCreateCommand);
      db.execute(dogsTableCreateCommand);
      return null;
    },
  );
}
```

Further, we have the insert, update, delete, and many other functions available in the database.

```
void addNewDog(Pet pet) {
  database.insert(dogsTableName, pet.toMap());
}

Future<List<Pet>> fetchDogs() async {
  List<Pet> dogssList = [];
  final list = await database.query(dogsTableName);
  for (var i = 0; i < list.length; i++) {
    dogssList.add(Pet.fromMap(list[i]));
  }
  return dogssList;
}

void updateDog(Pet pet) {
  database.update(dogsTableName, pet.toMap(),
      where: 'id = ?', whereArgs: [pet.id]);
}

void deleteDog(String dogId) {
  database.delete(dogsTableName, where: 'id = ?', whereArgs: [dogId]);
}
```

We can also perform all the functions manually by using the database.execute() and pass the SQL command manually but the package provides its helper methods so should we consume them.

where clause and **where arguments** are also important parts of interacting with a database because while updating, deleting, or fetching a specific entity you have to give it a where clause which includes the argument name and ? operator, and where argument contains the value which will replace the ? at runtime so the database can operate on the specific record containing the value (where argument) at the specific attribute (where clause).

```
void addDogIfNotExit(Pet pet) async {
  final list = await database
      .query(dogsTableName, where: 'id = ?', whereArgs: [pet.id.toString()]);
  list.isNotEmpty ? updateDog(pet) : addNewDog(pet);
}
```

32. Provider

Provider is simply a modified, enhanced, and capsulated form of Inherited Widget.
For the simple implementation of the provider wrap the widget with the provider. The provider will be available in the child hierarchy of that widget.
You can use the MultiProvider widget if you want more than one provider, remember Provider do type-based searching so you cannot add multi-providers of the same type.

First, import the provider library,

```
provider: ^6.1.2
```

Now wrap the target widget with the provider or multi-provider widget as required. And create the object of the selected type.

```
@override
Widget build(BuildContext context) {
  return Provider<String>(
    create: (context) => 'You have pushed the button this many times:',
    child: MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ), // ThemeData
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    ), // MaterialApp
```

```dart
@override
Widget build(BuildContext context) {
  return MultiProvider(
    providers: [Provider(create: (context) => 'You have pushed the button this many times:',)],
    child: MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ), // ThemeData
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    ), // MaterialApp
  ); // MultiProvider
}
}
```

Now we have registered the Provider on the top of the tree. So we can access this provider anywhere inside the tree. Now there are multiple ways to access this Provider down the tree. Firstly we will see the traditional way as inherited widgets are consumed.

```dart
    ), // AppBar
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          Text(Provider.of<String>(context)),
          Text(
            '$_counter',
            style: Theme.of(context).textTheme.headlineMedium,
```

This provider gives the object from the top by type-based searching as well as listening to that object so whenever that object gets modified this provider rebuilds the context. But if you want it to only provide you the data once. You can make listen false.

```dart
    ), // AppBar
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          Text(Provider.of<String>(context,listen: false)),
          Text(
            '$_counter',
            style: Theme.of(context).textTheme.headlineMedium,
```

Another way to do the same is context.watch() for a provider with listenable true and context.read() for a provider with listenable false. Here is how to do this

```
), // AppBar
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(context.read<String>()),
      Text(
        '$_counter',
        style: Theme.of(context).textTheme.headlineMedium,
```

```
), // AppBar
body: Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(context.watch<String>()),
      Text(
        '$_counter',
        style: Theme.of(context).textTheme.headlineMedium,
```

Also, we can use context.select<T>() to further select a particular part of the object.

Another way to do this is by using a Consumer Widget. The Consumer widget will rebuild the widget when the value updates.

```
child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: <Widget>[
    Consumer<String>(
      builder: (context, value, child) {
        return Text(value);
      },
    ), // Consumer
    Text(
```

Same as context.select() we can use the selector widget as well. It will rebuild the selector closure so the cost of rebuilding the entire widget will be reduced.

Further there is another widget **ChangeNotifierProvider** in this provider we can pass an instance of a class which have implemented **ChangeNotifier** class.
Now all the methods in this class which required UI to rebuild after mutating the state will have to call a function **notifyListener()** (This method get inherited from ChangeNotifier) or the data which will potentially modify the state.

Consuming this provider is same as consuming the other providers.

33. Riverpod
34. Bloc
35. GetX

Responsiveness

(a) Using MediaQuery.sizeOf()
(b) Using Flex via Flexible or Expanded
(c) Using LayoutBuilder Widget

Adaptiveness

(a) Using MediaQuery.orientationOf()
(b) Using OrientationBuilder()

Platform Specific Code

(a) Using Platform class